

COMP0085: Approximate Inference

Xu Chen

April 7, 2026

Contents

1	Graphical Models and Conditional Independence	7
1.1	Factor Models	7
1.2	Conditional Independence	7
1.3	Neighbourhood Definition	7
1.4	Markov Property	7
1.5	Markov Blanket	7
1.6	Undirected Graphical Models	8
1.7	Cliques and Maximal Cliques	8
1.8	Directed Acyclic Graphical (DAG) Model	8
1.9	Bayes-Ball Algorithm	8
1.10	Markov Boundary for X in DAG	8
1.11	Equivalence between Graphs and Families of Distributions	9
1.12	Adding Edges Changes Expressivity	9
1.13	Polytrees	9
1.14	Undirected Trees	9
1.15	Single Rooted Directed Trees	9
2	Polytrees, Message Passing, and Factor Models	10
2.1	Polytrees to Factor Models	10
2.2	Converting Undirected Trees to Rooted Directed Trees	10
2.3	Finding Marginals in Undirected Trees	10
2.4	Messages from Observed Leaf Nodes	11
2.5	Observed Internal Nodes	11
2.6	General Message Formulation	11
2.7	Belief Propagation for Latent Class Models	11
2.8	Belief Propagation for Polytrees	11
3	Junction Trees and Message Passing Algorithms	13
3.1	Converting Graphs While Preserving Distributions	13
3.2	Junction Tree Algorithm	13
3.3	Running Intersection Property	13
3.4	Message Passing on Junction Trees	13
3.5	Local and Global Consistency	14
3.6	Hugin Propagation Algorithm	14
3.7	Computational Complexity	15
3.8	Learning in Graphical Models	15

4	Model Selection and Bayesian Inference	17
4.1	Bayesian Evidence	17
4.2	Laplace Approximation	17
4.3	Bayesian Information Criterion (BIC)	17
4.4	Automatic Relevance Determination (ARD)	18
4.5	Bayesian Prediction	18
4.6	Bayesian Nonparametrics	18
4.7	Integrating Out Linear Parameters	18
4.8	Posterior Integration as Joint Gaussian	19
4.9	Covariance Kernel Learning	19
5	Gaussian Processes and Latent Variable Models	20
5.1	Gaussian Processes	20
5.2	Brownian Motion as a Gaussian Process	20
5.3	Gaussian Process Regression	20
5.4	Stationary Gaussian Processes	20
5.5	Radial Kernels	21
5.6	Independent Components Analysis (ICA)	21
5.7	Noiseless ICA and Infomax	21
5.8	Mutual Information	21
5.9	Conditional Entropy	22
5.10	Feedforward Models and Infomax Learning	22
6	Advanced Models: Boltzmann Machines	23
6.1	Kurtosis and Non-Gaussianity	23
6.2	Extended Kalman Filter (EKF)	23
6.3	Boltzmann Machines	23
6.4	Restricted Boltzmann Machines (RBM)	24
6.5	Latent Dirichlet Allocation (LDA)	24
7	Variational Inference Methods	26
7.1	Variational Approximation Setup	26
7.2	Variational Lower Bound	26
7.3	Factored Variational Inference	26
7.4	Mean-Field Approximations	26
7.5	Mean-Field for Boltzmann Machines	26
7.6	Mean-Field Approximation for Factorial HMMs	27
7.7	Structured Variational Inference	27
7.8	Variational Inference on DAGs	27
7.9	Variational Bayes	28
8	Complete Exponential Family VB and Advanced Methods	29
8.1	Complete Exponential Family VB	29
8.2	EM for MAP vs. Variational Bayesian EM	29
8.3	Variational Free Energy with Hyperparameters	30
8.4	ARD with Variational Bayesian Methods	30
8.5	Variational Sparse GP Approximations	30

9	Sigma-Point Filtering and Expectation Propagation	31
9.1	Sigma-Point Filter	31
9.2	Alternative KL Divergence Minimization	31
9.3	Expectation Propagation (EP)	31
9.4	EP Algorithm	32
9.5	Message Form in EP	32
9.6	Moment Matching in Exponential Families	32
10	Gaussian Process Approximations and Advanced EP Methods	33
10.1	Normal Gaussian Process	33
10.2	Exponential Family Factorization	33
10.3	Unnormalized Log-Sum Approximation	34
10.4	Unnormalized KL Divergence	34
10.5	Update Rule for Unnormalized Sites	34
10.6	Approximate Log-Likelihood	34
10.7	Gradient with Respect to Hyperparameters	34
10.8	Alpha Divergence	35
11	Loopy Inference and Loopy BP Methods	36
11.1	Dealing with Loops	36
11.2	Loopy BP as Message-Based Expectation Propagation	36
11.3	Loopy BP as Tree-Based Reparameterization	37
12	Bethe Free Energy and Convex Duality	39
12.1	Bethe Free Energy on Loopy Consistent Beliefs	39
12.2	Lagrangian Optimization of Bethe FE	39
12.3	Fixed Points of Loopy BP	40
12.4	Bethe FE as Second-Order Approximation	40
12.5	Log Partition Function	40
12.6	Negative Entropy and Duality	40
12.7	Legendre-Fenchel Conjugate	41
13	Convex Duality and Discrete MRF Inference	42
13.1	KL Divergence and Duality	42
13.2	Coupled Exponential Family Representations	42
13.3	Discrete Markov Random Fields	42
13.4	Non-Convexity of Bethe Entropy	43
13.5	Spanning Tree Decomposition	43
13.6	REINFORCE Trick	43
13.7	Recognition Models and Amortised Inference	44
14	Deep Learning and Parameterized Variational Inference	46
14.1	Wake-Sleep Learning Algorithm	46
14.2	Variational Autoencoders (VAE)	46
14.3	Reparameterization Trick	47
14.4	Importance-Weighted Variational Bounds	47
14.5	Normalising Flows	48

15 Diffusion Models and Score-Based Methods	49
15.1 Diffusion Model Overview	49
15.2 Variational Generative Model (Backward Model)	49
15.3 Recognition-Parametrized Model (RPM)	49

Introduction

These lecture notes cover approximate inference methods for probabilistic graphical models. The course develops techniques for computing with complex distributions where exact inference is intractable.

Course Structure:

1. **Foundation (Sections 1-3):** Graphical models, conditional independence, exact inference algorithms (polytrees, junction trees)
2. **Approximate Inference Methods (Sections 4-9):** Variational inference, expectation propagation, loopy belief propagation
3. **Theoretical Foundations (Sections 10-11):** Bethe free energy, convex duality, connection to statistical physics
4. **Applications and Extensions (Sections 12-15):** Model selection, Gaussian processes, Boltzmann machines, deep learning approaches

The core mathematical framework builds progressively: from exact inference on trees, to approximate inference on general graphs, to the theoretical justification via variational bounds and convex optimization.

Part I: Foundations — Graphical Models and Exact Inference

The foundation of this course rests on understanding graphical models and their induced conditional independence structures. We begin with exact inference algorithms on trees, which serve as the basis for understanding approximate inference on graphs with loops.

1 Graphical Models and Conditional Independence

1.1 Factor Models

Definition 1.1 (Factor Model). A factor model represents a probability distribution as a product of factors:

$$P(Z) = \frac{1}{Z} \prod_{i=1}^m f_i(Z_i)$$

where factors are functions of subsets of variables. Each factor f_i induces conditional dependencies between variables in its scope.

1.2 Conditional Independence

Definition 1.2 (Conditional Independence in Graphs). Variables X and Y are conditionally independent given V (written $X \perp Y|V$) if:

$$P(X, Y|V) = P(X|V)P(Y|V)$$

Equivalently, $P(X|Y, V) = P(X|V)$. In graphical models, this holds when every path between X and Y contains some variable in V (for undirected graphs or specific graph types).

1.3 Neighbourhood Definition

Remark 1.1 (Neighbourhood Relationship in Graphical Models). In factor models, variables are neighbours if they share a common factor, meaning there exists a factor that references both variables.

1.4 Markov Property

Definition 1.3 (Markov Property). Variable X is conditionally independent of all non-neighbours given its neighbours:

$$X \perp Y \mid \text{ne}(X), \quad \forall Y \notin \{\text{neighbours of } X\}$$

This is conditional independence of all non-neighbours given its neighbours.

1.5 Markov Blanket

Definition 1.4 (Markov Blanket). The Markov blanket (or Markov boundary) of variable X is a minimal set of variables M such that X is conditionally independent of all other variables given M . That is:

$$X \perp \{\text{all other variables}\} \mid M$$

In graphical models, the Markov blanket typically includes the variable's parents, children, and co-parents (parents of children).

1.6 Undirected Graphical Models

Definition 1.5 (Undirected Graphical Models). Markov networks (or undirected graphical models) represent factors as cliques on the graph. Neighbours in a Markov net share a factor.

Joint probability factors over the maximal cliques (or complete subgraphs) C of the graph.

1.7 Cliques and Maximal Cliques

Definition 1.6 (Cliques and Maximal Cliques). Cliques are fully connected subgraphs (every two nodes are connected). Maximal cliques are cliques not contained in other cliques (cannot be enlarged to add more vertices).

1.8 Directed Acyclic Graphical (DAG) Model

Definition 1.7 (Directed Acyclic Graph (DAG) Model). The joint probability factorizes:

$$P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i | X_{\text{pa}(i)}) \quad \text{pa}(i) \text{ are parents of node } i$$

These are Bayesian networks.

1.9 Bayes-Ball Algorithm

Remark 1.2 (Bayes-Ball Algorithm). The Bayes-Ball algorithm determines conditional independence in DAGs by asking: can a ball travel from X to Y without being blocked by conditioning set V ? This determines $X \perp Y | V$.

Rules:

- **Nodes not in V :** Balls pass through (either direction on chain)
- **Nodes in V :** Block balls traveling along edges involving two children or arrows into the node
- **At v-structures ($X \leftarrow Z \rightarrow Y$):** Block balls unless Z or its descendants are in V

Blocking set: A path from X to Y is blocked by V if:

- There exists a non-collider node C on the path with $C \in V$, or
- There exists a collider node C on the path with $C \notin V$ and no descendant of C is in V

If no unblocked path exists from X to Y given V , then $X \perp Y | V$.

1.10 Markov Boundary for X in DAG

Remark 1.3 (Markov Boundary in Directed Acyclic Graphs). In a DAG, the Markov boundary of X consists of: parents, children, and co-parents of children.

$$\text{MB}(X) = \text{pa}(X) \cup \text{ch}(X) \cup \text{pa}(\text{ch}(X))$$

This is the minimal set such that $X \perp (\text{all other variables}) | \text{MB}(X)$.

1.11 Equivalence between Graphs and Families of Distributions

Remark 1.4 (Graphs Induce Families of Distributions). A graph G induces a family of distributions by defining conditional independence statements (CIS). For any graph:

- **Directed graphs (DAGs):** $P(Z) = \prod_i P(X_i | \text{pa}(i))$ — global Markov implies factorization
- **Undirected graphs:** By Hammersley-Clifford theorem, positive distributions satisfying all global Markov properties factor as: $P(Z) = \frac{1}{Z} \prod_c f_c(Z_{C_c})$ where cliques C_c correspond to maximal complete subgraphs
- **Factor graphs:** Can represent families not capturable by undirected graphs alone (more expressive)

All conditional independence statements implied by a graph G are preserved in the corresponding family of distributions.

1.12 Adding Edges Changes Expressivity

Remark 1.5 (Effect of Adding Edges to Graphs). Adding edges to a graph G removes conditional independence assertions from the induced family:

$$G' = G + \text{edge} \Rightarrow \mathcal{P}_{G'} \supseteq \mathcal{P}_G$$

The new family is larger since we've removed constraints. This means denser graphs are more expressive but less constrained by conditional independence.

1.13 Polytrees

Definition 1.8 (Polytrees as Tree-Structured DAGs). Polytrees are tree-structured DAGs that may have more than one root.

$$P(Z) = \prod_i P(X_i | X_{\text{pa}(i)}) = \prod_i f_i(X_i) \quad C_i = \cup \text{pa}(i) \quad \text{and} \quad f_i(X_{C_i}) = P(X_i | X_{\text{pa}(i)})$$

1.14 Undirected Trees

Definition 1.9 (Undirected Trees and Pairwise Factors). For undirected tree, all maximal cliques are of size 2; factor graph has pairwise factors.

$$P(Z) = \frac{1}{Z} \prod_{e \in E} f_{uv}(X_i, Y_j)$$

1.15 Single Rooted Directed Trees

Definition 1.10 (Single Rooted Directed Trees). Single rooted directed tree can be written as a product of pairwise factors $\Rightarrow$ undirected tree.

$$P(Z) = P(X_r) \prod_{i \neq r} P(X_i | X_{\text{pa}(i)}) = \prod_{e \in E} f_{uv}(X_i, X_r)$$

2 Polytrees, Message Passing, and Factor Models

2.1 Polytrees to Factor Models

Remark 2.1 (Polytrees and Factor Models). A polytree (CDAG - directed acyclic graph) can be factorized as a factor model. A single rooted tree (CDAG) factorizes through pairwise factors via an undirected tree representation:

$$P(X) = P(X_i) \prod_{j \in \text{ne}(i)} P(X_j | X_i) \text{ from belief propagation}$$

An undirected tree can be obtained from the CDAG by treating edges as undirected.

2.2 Converting Undirected Trees to Rooted Directed Trees

Remark 2.2 (Root Selection and Directed Conversion). Choose an arbitrary node X_r to be the root and orient all arrows away from it. Then:

- Compute $P(X_i)$ and $P(X_i | X_j)$ relationships
- The joint distribution becomes:

$$P(X_i) = P(X_r) \prod_{i \in \text{pa}(i)} P(X_i | X_{\text{pa}(i)}) = P(X_r) \prod_i \frac{P(X_i, X_j)}{P(X_j)} = \frac{\prod_i \text{numerator products}}{Z}$$

where degree accounting: $\deg(i) = \# \text{ being parent} + \# \text{ being child}$

- For the rooted representation: $\deg(i) = \# \text{ being parent}$, leaving one as prior

2.3 Finding Marginals in Undirected Trees

Remark 2.3 (Marginal Computation via Message Passing). For an undirected tree with pairwise factorization:

$$P(X) = \frac{1}{Z} \prod_{i,j \in E} f_{ij}(X_i, X_j)$$

The marginal $P(X_i)$ is obtained by summing over all other variables. Using message passing, we compute:

$$P(X_i) = \sum_{X_{\setminus i}} P(X) \propto \sum_{X_{\setminus i}} \prod_{j \in \text{ne}(i)} f_{ij}(X_i, X_j)$$

This can be decomposed recursively. For each neighbour j of i , define the message:

$$M_{j \rightarrow i}(X_i) = \sum_{X_j} f_{ij}(X_i, X_j) \prod_{k \in \text{ne}(j) \setminus i} f_{jk}(X_j, X_k)$$

The marginal factorizes as:

$$P(X_i) \propto \prod_{j \in \text{ne}(i)} M_{j \rightarrow i}(X_i)$$

Similarly, pairwise marginals are:

$$P(X_i, X_j) = \sum_{X_{\setminus i,j}} P(X) \propto f_{ij}(X_i, X_j) M_{i \rightarrow j}(X_j) M_{j \rightarrow i}(X_i)$$

2.4 Messages from Observed Leaf Nodes

Remark 2.4 (Conditioning on Observations). When observing evidence, messages are conditioned rather than marginalized:

- To compute $P(X_i)$ with no observations:

$$M_{a \rightarrow i}(X_i) = \sum_{X_a} f_{aX}(X_a, X_i) \propto \sum_{X_a} P(X_a|X_i)P(X_i)$$

- To compute $P(X_i|X_a = a)$ when X_a is observed:

$$M_{a \rightarrow i}(X_i) = f_{aX}(X_a = a, X_i) = \sum_{X_a} \delta_a(X_a) f_{aX}(X_a, X_i) \propto P(X_a = a|X_i)P(X_i)$$

2.5 Observed Internal Nodes

Remark 2.5 (Partitioning by Observed Nodes). When an observed node is in the interior of the tree, it partitions the graph, and messages propagate independently:

$$M_{b \rightarrow j} = f_{bX}(X_b = b, X) \quad \text{vs} \quad M_{a \rightarrow *} = f_{aX}(X_a = b, X)$$

2.6 General Message Formulation

Remark 2.6 (Messages and Likelihood Scaling). Messages $M_{i \rightarrow j}(X_j)$ are proportional to the likelihood of observed variables (O) within the subtree $T_{i \rightarrow j}$, possibly scaled by prior factors:

$$M_{i \rightarrow j}(X_j) \propto P(X_{T_{i \rightarrow j}} \cap [O]|X_j)P(X_j)$$

where the message encodes belief about X_j given all evidence in the subtree rooted at i (when viewing j as the root).

2.7 Belief Propagation for Latent Class Models

Definition 2.1 (BP Updates for Latent Class Models). For models with latent class variables, belief propagation follows:

- Define $\alpha_i(k) = P(X_1, \dots, X_i, \theta = k)$
- Forward pass: $\alpha_{i+1}(k) = P(X_{i+1}|\theta = k) \sum_k \alpha_i(k) \beta_i(k)$
- Backward pass: compute messages and marginals

2.8 Belief Propagation for Polytrees

Remark 2.7 (Factor Nodes in Polytrees). Associated undirected tree contains loops created by non-binary latent classes. We can treat factors as message-passing nodes:

$$P(X) = \prod_i \text{node}_i(X_i) \prod_a \text{factor}_a(X_a)$$

Messages pass between variable nodes and factor nodes:

- **Incoming** (node $\rightarrow$ factor):

$$M_{i \rightarrow a}(X_i) = \sum_{X_i} P(X_i | X_{\text{pa}(i)}) \prod_{b \neq a} M_{b \rightarrow i}(X_i)$$

- **Outgoing** (factor $\rightarrow$ node):

$$M_{a \rightarrow i}(X_i) = \frac{1}{Z_i} f_a(X_i, X_a) \prod_{j \in \text{ne}(a) \setminus i} M_{j \rightarrow a}(X_j)$$

This allows exact inference on polytree-structured graphs through efficient message passing.

3 Junction Trees and Message Passing Algorithms

3.1 Converting Graphs While Preserving Distributions

Remark 3.1 (Equivalence Under Graph Transformation). A transformation from graph G to an easier-to-handle graph G' is valid only if $P(X)$ can also be represented by G' . The critical principle:

Only remove conditional independence assertions; never add them.

3.2 Junction Tree Algorithm

Remark 3.2 (Construction Steps for Junction Trees). The junction tree algorithm transforms a factor graph into a tree structure suitable for efficient inference:

1. **DAGs to factor graphs:** Convert $P(X) = \prod P(X_i | X_{\text{pa}(i)}) = \prod f_i(X_{\alpha_i})$ where $\alpha_i = i \cup \text{pa}(i)$ and $P(X_r)$ is absorbed into an adjacent factor.
2. **Incorporating observations:** For observed data O , the distribution becomes $P(X|O) \propto P(X, O) = P(X) \mathbf{1}_S(O)$. Add singleton factors with δ -functions: $P(X, O) = \prod [f_a] f_b(O) f_c(C)$
3. **Moralization:** Convert factor graph to undirected graph by replacing each factor with an undirected clique. Construct potentials on maximal cliques by multiplying factor potentials, ensuring each appears exactly once.
4. **Triangulation:** Add edges so every cycle of length ≥ 4 has at least one chord. Use either:
 - *Minimum deficiency search:* Choose variable inducing fewest new edges
 - *Maximum cardinality search:* Choose variable with most previously visited neighbours

The resulting chord graph must be direct (undirected).

5. **Clique graph to junction tree:** Create nodes representing cliques; edges represent separators (intersections of cliques). This forms a tree structure.

3.3 Running Intersection Property

Definition 3.1 (Running Intersection Property). If two cliques contain variable X , all cliques and separators on the path between them must also contain X . This property is guaranteed if and only if the graph is chordal (triangulated).

3.4 Message Passing on Junction Trees

Remark 3.3 (Junction Tree Message Passing). In a junction tree, nodes represent maximal cliques C_i , and edges represent separators. Messages pass between adjacent cliques, implementing distributional updates.

For a simple chain: $(ABC) - (BC) - (BCD)$ with separators (B, C) and (B, C, D) , the message from i to j is:

$$M_{i \rightarrow j}(X_{s_{ij}}) = \sum_{X_{\alpha_i} \setminus X_{s_{ij}}} f(C_i) \prod_{l \in \text{neigh}(i) \setminus j} M_{l \rightarrow i}(X_{s_{il}})$$

The marginals are computed as:

$$P(X_{\alpha_i}) \propto f_i(X_{\alpha_i}) \prod_{l \in \text{neigh}(i)} M_{l \rightarrow i}(X_{s_{il}})$$

For separator marginals:

$$P(X_{s_{ij}}) \propto M_{i \rightarrow j}(X_{s_{ij}}) M_{j \rightarrow i}(X_{s_{ij}})$$

3.5 Local and Global Consistency

Remark 3.4 (Consistency Conditions). If $g_i(X_{c_i})$ and $r_{ij}(X_{s_{ij}})$ are distributions satisfying the marginal consistency:

$$\sum_{X_{\alpha_i} \setminus X_{s_{ij}}} g_i(X_{c_j}) = r_{ij}(X_{s_{ij}})$$

then

$$P(X) = \frac{\prod g_i(X_{c_i})}{\prod_{l \in \text{neigh}(i)} r_{ij}(X_{s_{ij}})}$$

is also a valid distribution. The consistency is maintained because each separator potential must be counted only once. We have:

$$g_i(X_{c_i}) = \sum P(X), \quad b_{ij}(X_{s_{ij}}) = \sum P(X)$$

3.6 Hugin Propagation Algorithm

Remark 3.5 (Hugin Propagation Updates). The Hugin algorithm maintains consistency through iterative updates:

- **Initialization:** Set $\psi_i(X_{c_i}) \propto f_i(X_{c_i})$ and $r_{ij}(X_{s_{ij}}) \propto 1$ (unfactored); then $P(X) \propto \frac{\prod \psi_i(X_{\alpha_i})}{\prod r_{ij}(X_{s_{ij}})}$
- **Update for $i \rightarrow j$:**

$$\psi'_i(X_{s_{ij}}) = \sum q'_i(X_{c_i}), \quad r'_{ij}(X_{s_{ij}}) = \frac{\psi'_i(X_{s_{ij}}) r_{ij}(X_{s_{ij}})}{r_{ij}^{\text{old}}(X_{s_{ij}})}$$

- $P(X)$ remains unchanged after updates — local consistency is maintained throughout.

3.7 Computational Complexity

Remark 3.6 (Time and Memory Costs). The computational cost of message passing is proportional to:

$$O\left(\sum_i |X_{c_i}| \cdot |X_{c_i}|\right) = O\left(\sum_i \text{number of combinations of states in clique } i\right)$$

This scales exponentially with the clique size, making triangulation heuristics important for finding small-treewidth decompositions.

3.8 Learning in Graphical Models

Remark 3.7 (Parameter Estimation via Expectation-Maximization). For learning shared parameters θ , the likelihood is:

$$\ell = P(X_{\text{obs}}|\theta) = \sum_{X_{\text{latent}}} P(X_{\text{obs}}, X_{\text{latent}}|\theta)$$

The log-likelihood can be decomposed:

$$\log P(X_{\text{obs}}, X_{\text{latent}}|\theta) = \sum_j \log f_i(X_{c_j}|\theta) - \log Z(\theta) + H(\theta)$$

where $Z(\theta)$ is the partition function. The EM algorithm iterates between:

- **E-step:** Compute expected sufficient statistics under the posterior
- **M-step:** Update θ to maximize expected log-likelihood

Part IV: Applications and Extensions

The approximate inference methods developed in Part II have broad applicability. We now examine concrete applications: model comparison and selection, Gaussian processes as a nonparametric approach to regression, Boltzmann machines as classical models of distributed computation, and modern deep learning approaches that combine neural networks with variational inference.

4 Model Selection and Bayesian Inference

4.1 Bayesian Evidence

Definition 4.1 (Bayesian Model Evidence). For comparing models m , the evidence is:

$$P(D|m) = \int_{\theta_m} P(D|\theta_m, m)P(\theta_m|m)d\theta_m$$

In conjugate prior settings:

$$P(D|m) = \frac{\text{posterior value}}{\text{prior volume}}$$

4.2 Laplace Approximation

Remark 4.1 (Laplace Approximation for Model Evidence). To approximate $\log P(D, \theta_m|m)$, expand to second order around the maximum θ^* (the posterior mode):

$$\log P(D, \theta_m|m) \approx \log P(D, \theta_m^*|m) + \nabla \log P(D, \theta_m^*|m)(\theta_m - \theta_m^*) + \frac{1}{2}(\theta_m - \theta_m^*)^T A(\theta_m - \theta_m^*)$$

where $A := -\nabla^2 \log P(D, \theta_m^*|m)$ is the negative Hessian evaluated at the posterior mode.

At the mode, $\nabla \log P(D, \theta_m^*|m) = 0$, so:

$$P(D|m) = \int e^{\log P(D, \theta_m|m)} d\theta_m \approx P(D|\theta_m^*, m)P(\theta_m^*|m)(2\pi)^{d/2}|A|^{-1/2}$$

This approximates the posterior with a Gaussian.

4.3 Bayesian Information Criterion (BIC)

Remark 4.2 (Asymptotic Approximation with BIC). The model evidence becomes:

$$\log P(D|m) \approx \log P(\theta_m^*|m) + \log P(D|\theta_m^*, m) + \frac{d}{2} \log 2\pi - \frac{1}{2} \log |A|$$

As $N \rightarrow \infty$, we have $A \rightarrow NA_0 + \text{const}$ where A_0 is a fixed positive definite matrix. Thus:

$$\log |A| \rightarrow \log |NA_0| = d \log N + \log |A_0|$$

Keeping only terms varying with N :

$$\log P(D|m) \approx \log P(D|\theta_m^*, m) - \frac{d}{2} \log N$$

This leads to model selection criteria:

- **BIC**: $\text{BIC}' = d \log N - 2 \log P(D|\theta_m^*, m)$
- **AIC**: $\text{AIC} = 2d - 2 \log P(D|\theta_m^*, m)$

We can use ML estimate of θ instead of MAP for these approximations.

4.4 Automatic Relevance Determination (ARD)

Remark 4.3 (Hyperparameter Learning via ARD). For prior $\alpha \propto N(0, C)$ with $\psi_i \sim N(W^T X_i, \sigma^2)$, and posterior on W with $C^{-1} = \text{diag}(\alpha)$:

$$\mathcal{E}(C, \sigma^*) = \int P(Y|X, W, C, \sigma^*) P(W|C) dW$$

Taking derivatives with respect to hyperparameters yields sparse solutions through Automatic Relevance Determination (ARD).

The updates are:

$$\alpha_i^{\text{new}} = \frac{\gamma_i}{w_i^2}, \quad (w_i^{\text{new}})^T = \frac{\gamma_i(X_i - w_i^T X)}{N - \sum(1 - E_w[w_i])\alpha_i}$$

These sparse solutions effectively perform variable selection during learning.

4.5 Bayesian Prediction

Remark 4.4 (Prediction Averaging). Rather than using point estimates of parameters, Bayesian prediction averages over the posterior:

For unsupervised learning:

$$P(x|D, m) = \int P(x|\theta, m) P(\theta|D, m) d\theta$$

For supervised learning:

$$P(y|x, D, m) = \int P(y|x, \theta, m) P(\theta|D, m) d\theta$$

The integral naturally favours simpler models through an Occam-factor, since complex models with low likelihood contribute less to the average.

4.6 Bayesian Nonparametrics

Remark 4.5 (Nonparametric Models). Bayesian nonparametric models have no explicit parameters after marginalization. For instance, a Gaussian process defines a prior over functions directly.

4.7 Integrating Out Linear Parameters

Remark 4.6 (Linear Regression with Integrated Weights). If $w \sim N(0, \tau^2 I)$ and $\psi_i \sim N(W^T X_i, \sigma^2)$, integrating w yields:

$$y|X, D \sim N(\bar{w}^T X, X \Sigma_w X^T + \sigma^2)$$

If we integrate w with a kernel representation:

$$\begin{bmatrix} y \\ y_* \end{bmatrix} \Big| X, x \sim \mathcal{N} \left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} K_{xx} & K_{xx*} \\ K_{x*x} & K_{x*x*} \end{bmatrix} \right)$$

The conditional predictive distribution is:

$$y|Y, X, x \sim \mathcal{N}(K_{xx*} K_{xx}^{-1} Y, K_{x*x*} - K_{x*x} K_{xx}^{-1} K_{xx*})$$

This explains the regression prediction through kernel properties.

4.8 Posterior Integration as Joint Gaussian

Remark 4.7 (Gaussian Process Conditioning). Posterior integration with a kernel is equivalent to conditioning a joint Gaussian with that kernel (parameter-free approach).

4.9 Covariance Kernel Learning

Definition 4.2 (Covariance Kernel K). For a covariance kernel K , the conditional distribution is:

$$y|X, K \sim \mathcal{N}(0, K_{xx})$$

Optimization proceeds via gradient ascent in $\log P(X|X, K)$.

5 Gaussian Processes and Latent Variable Models

5.1 Gaussian Processes

Definition 5.1 (Gaussian Process (GP)). A Gaussian process is a stochastic process on $\mathbb{R}$ indexed by $x \in \mathcal{X}$, where any finite subset of which have Gaussian distributions. A GP is fully specified by a kernel function $k(\cdot, \cdot)$ and mean function $m(\cdot)$:

$$f(x) \sim \text{GP}(m(\cdot), k(\cdot, \cdot))$$

For a set of points $\{x_1, \dots, x_n\}$, the function values form a vector:

$$\mathbf{f} = [f(x_1), f(x_2), \dots, f(x_n)]^T \sim \mathcal{N}(m(x), K_{xx})$$

where $k(\mathbf{x}_i, \mathbf{x}_j)$ are the covariance kernel evaluations.

5.2 Brownian Motion as a Gaussian Process

Definition 5.2 (Brownian Motion). Brownian motion is a Gaussian process with kernel function:

$$k(s, t) = \sigma^2 \min(s, t)$$

where σ^2 is the variance coefficient per unit time (diffusion coefficient) of the process.

5.3 Gaussian Process Regression

Remark 5.1 (GPR Model and Inference). Standard regression using a GP prior:

- **GP prior:** $f(\cdot) \sim \text{GP}(0, k(\cdot, \cdot))$ implies $f(\mathbf{x}) \sim \mathcal{N}(0, K_{xx})$
- **Observation model:** $y_i | x_i, f \sim \mathcal{N}(f(x_i), \sigma^2)$, so $Y \sim \mathcal{N}(0, K_{xx})$ with $\tilde{K}_{xx} = K_{xx} + \sigma^2 I$
- **Posterior on latent f :** Also a GP:

$$f(\cdot) | X, Y \sim \text{GP} \left(K_{x*} (\tilde{K}_{xx})^{-1} Y^T, k(\cdot, \cdot) - K_{x*} (\tilde{K}_{xx})^{-1} K_{*x} \right)$$

- **Prediction with observation noise:**

$$y | X, Y, x \sim \mathcal{N} \left(K_{xx'} \tilde{K}_{xx}^{-1} Y, K_{xx'} - K_{xx'} \tilde{K}_{xx}^{-1} K_{xx'} + \sigma^2 \right)$$

5.4 Stationary Gaussian Processes

Definition 5.3 (Translation-Invariant Kernels). A Gaussian process with translation-invariant covariance kernel is stationary: if $f(t) \sim \text{GP}(0, k)$, then $f(\cdot - x) \sim \text{GP}(0, k)$ for any fixed x . This means the covariance depends only on the relative distance between points.

5.5 Radial Kernels

Definition 5.4 (Radial and Radially Symmetric Kernels). A covariance kernel is radial or radially symmetric if:

$$k(x, x') = h(\|x - x'\|)$$

A GP with a radial covariance kernel exhibits stationarity with respect to translations, rotations, and reflections of the input space, capturing symmetry in the underlying data-generating process.

5.6 Independent Components Analysis (ICA)

Definition 5.5 (ICA Model). ICA decomposes observed data as a linear mixture of independent non-Gaussian components:

$$X = \Lambda Z + \varepsilon$$

where $X \in \mathbb{R}^d$, $Z \in \mathbb{R}^k$ contains non-Gaussian independent components, and ε is typically zero or small noise. The model is identifiable even when $k > d$ (overcomplete) or $k \leq d$ (undercomplete) provided components are sufficiently non-Gaussian, making it tractable where PCA fails.

5.7 Noiseless ICA and Infomax

Remark 5.2 (Noiseless (Infomax) ICA). In the noiseless case, $x = \Lambda z$, we use the unmixing matrix $W = \Lambda^{-1}$ to recover:

$$z = Wx$$

The likelihood incorporates the change-of-variables Jacobian:

$$P(x|W) = \prod_k p_k(W_k x) |W|, \quad P(z) = \prod_k p_k(z_k)$$

The log-likelihood becomes:

$$\log P(x) = \log |W| + \sum_k \log p_k(W_k x)$$

Standard gradient descent update: $\Delta W \propto \nabla_W \log P(x) = (W^T)^{-1} + \nabla_W \sum \log p_k(W_k x)$

A better approach uses natural/covariant gradient which is more efficient for this manifold:

$$\Delta W \propto \nabla_W \log P(x) (W^T W) = W + g(z) z^T W$$

Note: EM cannot be used since there is no randomness (posterior is a Dirac delta function), and A is screened from S in the M-step, making it circular.

5.8 Mutual Information

Definition 5.6 (Mutual Information). The mutual information between random variables X and Y measures their statistical dependence:

$$I(X; Y) = \text{KL}[p_{xy} \| p_x p_y] = H(X) - H(X|Y) = H(Y) - H(Y|X)$$

where H denotes entropy, measuring the average reduction in uncertainty about one variable from observing the other.

5.9 Conditional Entropy

Definition 5.7 (Conditional Entropy). The conditional entropy of Y given X is:

$$H(Y|X) = - \sum_x \sum_y p(x, y) \log \frac{p(x, y)}{p(x)}$$

This measures the average remaining uncertainty in Y after observing X .

5.10 Feedforward Models and Infomax Learning

Remark 5.3 (Infomax Principle for Learning). For a feedforward model $\tilde{z} = f_i(z_i)$ where f_i is a squashing function (e.g., sigmoid: $f_i(-\infty) = 0$, $f_i(+\infty) = 1$), the optimal learning objective is:

$$\operatorname{argmax}_W I(X; Z_i) = \operatorname{argmax}_W [H(Z) - H(Z|X)]$$

Because $f(W; x)$ is deterministic (given x , z is fully determined), we have $H(Z|X) = 0$.

The optimization thus becomes:

- **Maximize** $H(Z)$: Make the output distribution as uniform as possible over $[0, 1]$
- **Constraint**: The sources must become statistically independent as possible

This simultaneously maximizes information transmission through the network and encourages statistical independence in the latent representation.

6 Advanced Models: Boltzmann Machines

6.1 Kurtosis and Non-Gaussianity

Remark 6.1 (Kurtosis as Non-Gaussianity Measure). Kurtosis measures the non-Gaussianity of a source:

$$\text{Kurtosis} = \frac{E[(x - \mu)^4]}{E[(x - \mu)^2]^2} - 3$$

A key insight from the Central Limit Theorem: linear mixtures of independent non-Gaussian sources tend toward a more Gaussian distribution, driving kurtosis toward 0. This property makes kurtosis useful for identifying directions that maximize non-Gaussianity in ICA.

6.2 Extended Kalman Filter (EKF)

Definition 6.1 (Extended Kalman Filter). For nonlinear state-space models with nonlinear dynamics and observation functions, the Extended Kalman Filter approximates the posterior by linearizing around the current estimate:

$$\begin{aligned} Z_{t+1} &= f(Z_t^*, u_t) + \left. \frac{\partial f}{\partial z} \right|_{\hat{z}_t} (Z_t - \hat{Z}_t^*) + W_t \\ X_t &= g(\hat{Z}_t^*, u_t) + \left. \frac{\partial g}{\partial z} \right|_{\hat{z}_t} (Z_t - \hat{Z}_t^*) + V_t \end{aligned}$$

The linearization enables approximate inference in nonlinear systems, with Kalman filter-style alternation between prediction and update steps.

6.3 Boltzmann Machines

Definition 6.2 (Boltzmann Machine). An undirected graphical model over binary variables $S_i \in \{0, 1\}$ where some variables are hidden and some are observed:

$$P(S|W, b) = \frac{1}{Z} \exp \left[\sum_{i < j} W_{ij} S_i S_j - \sum_i b_i S_i \right]$$

where $W_{ij} = W_{ji}^T$. The energy function is:

$$H(S) = - \sum_i W_{ij} S_i S_j + \sum_i b_i S_i \quad \text{such that} \quad P(S) = \frac{1}{Z} e^{-H(S)}$$

Inference requires computing moments under both clamped (observed) and unclamped (full model) distributions:

$$\langle S^m P(S|S^v, W, b) \rangle \quad \text{and} \quad \langle S^H C H^T P(S|S^v, W, b) \rangle$$

Learning via EM maximizes:

$$\log P(C^N H^v | W, b) = \sum \left[W_{ij}^T S_i S_j - \sum b_i S_i - \log Z \right]$$

The M-step for weight updates separates into clamped expectations $\langle S_i S_j \rangle_c$ (from observed data) and unclamped expectations $\langle S_i S_j \rangle_u$ (from the full model).

6.4 Restricted Boltzmann Machines (RBM)

Definition 6.3 (RBM Architecture). A bipartite Boltzmann machine where $W_{ij} = 0$ for any two visible nodes or any two hidden nodes. This factorization enables efficient inference:

$$P(S^v|S^H) = \prod_{i \in v} \text{Bernoulli} \left(\sigma \left(\sum_{j \in H} W_{ij} S_j - b_i \right) \right)$$
$$P(S^H|S^v) = \prod_{j \in H} \text{Bernoulli} \left(\sigma \left(\sum_{i \in v} W_{ij} S_i - c_j \right) \right)$$

where σ is the sigmoid function. Unclamped samples can be generated efficiently using block Gibbs sampling by alternately sampling visible and hidden layers.

6.5 Latent Dirichlet Allocation (LDA)

Definition 6.4 (LDA Model for Topic Modeling). LDA is a generative model for document collections with latent topics:

- **Objects:** W possible words, D documents, K topics
- **Topic distribution per document:** $\theta_d | \alpha \sim \text{Dirichlet}(\alpha, \dots, \alpha)$ (length K)
- **Word distribution per topic:** $\phi_k | \beta \sim \text{Dirichlet}(\beta, \dots, \beta)$ (length W)
- **Topic assignment:** $z_{id} | \theta_d \sim \text{Discrete}(\theta_d)$ for the i -th word in document d
- **Word identity:** $x_{id} | z_{id}, \phi_{z_{id}} \sim \text{Discrete}(\phi_{z_{id}})$ given the assigned topic

The probability of word w in document d integrates over all topics:

$$P(x_w = w | d) = \sum_{k=1}^K \theta_{dk} \phi_{kw}$$

This model captures both document-topic and topic-word distributions, enabling unsupervised discovery of latent topics.

Part II: Approximate Inference Methods

When exact inference becomes intractable (as with loopy graphs or continuous variables), we resort to approximate methods. The following sections develop the three main paradigms: variational inference, expectation propagation, and loopy belief propagation. All fundamentally rely on minimizing or bounding divergences between the true and approximate posteriors.

7 Variational Inference Methods

7.1 Variational Approximation Setup

Definition 7.1 (Variational Approximation in EM). In variational approximation, if $P(Z|X, \theta^m) \in \mathcal{Q}$ where $\mathcal{Q}$ is a tractable class of approximations, then θ^m is a fixed point of the variational algorithm.

7.2 Variational Lower Bound

Definition 7.2 (Free Energy and ELBO). The variational free energy is:

$$\mathcal{F}(q, \theta) = \langle \log p(x, z|\theta) \rangle_{q(z)} + H(q) = \log P(X|\theta) - \text{KL}[q||p(z|x, \theta)]$$

The E-step and M-step become:

- **E-step:** $q^*(z) = \arg \max_q \mathcal{F}(q, \theta)$ (optimize q given current θ)
- **M-step:** $\theta^* = \arg \max_{\theta} \mathcal{F}(q, \theta)$ (optimize θ given current q)

7.3 Factored Variational Inference

Remark 7.1 (Factored Variational E-step). The most common factorization assumes $\mathcal{Q} = \{q : q(z) = \prod_i q_i(z_i)\}$ where the z_i form disjoint sets.

In the i -th step of the E-phase, we maximize $\mathcal{F}(q, \theta)$ with respect to $q_i(z_i)$ only, keeping other q_j and parameters fixed:

$$q_i^*(z_i) = \arg \max_{q_i} \mathcal{F}(\dots)$$

This iterates until convergence. The update rule is:

$$q_i(z_i) \propto \exp \langle \log p(x, z|\theta) \rangle_{\prod_{j \neq i} q_j}$$

This coordinate-ascent approach ensures the free energy monotonically increases at each update.

7.4 Mean-Field Approximations

Definition 7.3 (Mean-Field Factorization). Mean-field approximations assume complete factorization: $q(z) = \prod_i q_i(z_i)$ where each z_i is updated independently. Each variable receives only its marginal posterior expectation from its neighbours.

7.5 Mean-Field for Boltzmann Machines

Remark 7.2 (Mean-Field Updates in BMs). For a Boltzmann machine with $P(z, s) = \frac{1}{Z} \exp(\sum w_{ij} s_i s_j + \sum b_i s_i)$, the mean-field update for binary variables is:

$$\log P(x, z|\theta)_{\text{mean}} = \frac{1}{2} w_{ij} \langle s_i \rangle \langle s_j \rangle + \frac{1}{2} b_i \langle s_i \rangle$$

For $S_i \in \{+1, -1\}$ coding, we have $S_i = \sum W_{ij} \langle s_j \rangle$ (treating expectations as fixed parameters).

7.6 Mean-Field Approximation for Factorial HMMs

Remark 7.3 (Factored HMM Variational Updates). For systems with multiple independent chains:

$$q(S_t^{(m)}) = \prod_m q_t^{(m)}(S_t^{(m)})$$

The update for chain m is:

$$q_t^{(m)} \propto \exp \left[\left\langle \log P(S_t^{(m)}, X_{1:T}) \right\rangle_{\prod_{m' \neq m} q_t^{(m')}} \right]$$

This decomposes inference into parallel updates across independent components, dramatically reducing computational complexity compared to the full factorial HMM posterior.

7.7 Structured Variational Inference

Remark 7.4 (Structured Mean-Field for Complex Models). Rather than fully factored approximations, structured variational inference uses more complex factorizations that capture important dependencies:

$$\psi(S_t^{(m)}) = \prod_m \psi^{(m)}(S_t^{(m)})$$

Each component $\psi^{(m)}$ can capture local dependencies within that chain while remaining independent between chains:

$$\psi^{(m)}(S_t^{(m)}) \propto \exp \left[\sum_{\ell} \log P(S_t^{(m)} | S_{\ell}^{(m)}) + \sum P(X_t | S_t^{(m)}) \right]$$

This balances computational tractability with improved approximation quality.

7.8 Variational Inference on DAGs

Remark 7.5 (Message Passing for DAG Variational Inference). For directed acyclic graphs $P(X, z) = \prod P(V_k | \text{pa}(V_k))$ with factored assumption $Q(z) = \prod q_i(z_i)$ for disjoint sets $\{z_i\}$:

$$\log q_i^*(z_i) = \left\langle \sum_k \log P(V_k | \text{pa}(V_k)) \right\rangle_{q_m(z)} + \text{const}$$

This separates into:

$$= \sum \langle \log P(z_i | \text{pa}(z_i)) \rangle_{q_m(z)} + \sum \langle \log P(Q_j | \text{pa}(j)) \rangle$$

where the first sum covers parent messages and the second covers children and their parents. Each node receives messages from its Markov boundary, enabling efficient message-passing inference algorithms.

7.9 Variational Bayes

Definition 7.4 (Variational Bayes Algorithm). Variational Bayes extends mean-field to jointly optimize over both latent variables and parameters:

$$\log P(x|\mathcal{M}) = \log \int \int P(x, z|\theta, \mathcal{M})P(\theta|\mathcal{M})dzd\theta$$

Using the factorization $Q(z, \theta) = Q_z(z)Q_\theta(\theta)$, the evidence lower bound (ELBO) is:

$$\mathcal{F}(Q_z, Q_\theta) = \int \int \log \frac{P(x, z, \theta|\mathcal{M})}{Q_z(z)Q_\theta(\theta)} dzd\theta$$

The variational algorithm alternates:

- **E-like step:** $Q_z^*(z) \propto \exp\langle \log P(z|x, \theta) \rangle_{Q_\theta(\theta)}$
- **M-like step:** $Q_\theta^*(\theta) \propto \exp\langle \log P(x, z|\theta) \rangle_{Q_z(z)} P(\theta)$

The gap between the true log-evidence and the ELBO is:

$$\log P(x) - \mathcal{F}(Q_z, Q_\theta) = \text{KL}[Q_z(z)Q_\theta(\theta) \| P(z, \theta|x)]$$

8 Complete Exponential Family VB and Advanced Methods

8.1 Complete Exponential Family VB

Remark 8.1 (Complete Exponential Family Variational Bayes). When $Q_0(z)$ is also computed in the complete exponential family:

$$Q_0(z) \propto P(z) \exp \langle \log P(Z, X | \Theta) \rangle_{Q_z}$$

This can be written as:

$$\begin{aligned} &= \int Q(z) \ell(z)^Y \exp(q(\theta)/\Gamma) + q(\theta)^T \exp\left(\sum_{ij} T(Z_i, X_j)\right)_Q \\ &\propto \int Q(z, \Sigma) q(\theta)^Y \exp(q(\theta)/\Gamma) \end{aligned}$$

where we only need to track $\bar{b}$ and $\bar{\Sigma}$ with:

$$\bar{\Sigma} = \bigcup \Sigma, \quad \bar{\Sigma} = Z + \sum_{ij} \langle T(Z_i, X_j) \rangle_{Q_z}$$

Similarly:

$$\begin{aligned} S_{22}(\hat{Z}) &= \int Q_{02}(\hat{Z}) \cdot Q_{02}(\hat{Z}) \times 2 \exp \langle \log P(Z_i; X_i | b) \rangle \times \int q_u(z) \exp\left(\sum_{ij} b_{ij} ST(Z_i, X_j)\right) \\ &= P(\bar{Z}_i | X_i, \bar{\theta}(\theta)) \end{aligned}$$

This changes the natural parameter from $q(\theta)$ to $\langle b(\theta) \rangle_{\theta_Q}$.

8.2 EM for MAP vs. Variational Bayesian EM

Remark 8.2 (Comparison of EM Objectives). Two key variants of EM differ in their optimization goals and assumptions:

EM Optimization:

- Goal: Maximize $P(D|Z, \theta)$ with respect to θ
- E-step: Compute $Q_0(Z) \leftarrow P(Z|X, \theta)$
- M-step: $\theta \leftarrow \arg \max \int \log P(Z|Z, \theta; \theta) Q_0(Z) dZ$

Variational Bayesian EM:

- Goal: Maximize bound on $P(Z|m)$ with respect to Q_0
- VB-E-step: Compute $Q_0(Z) \leftarrow p(Z|X, \bar{\theta})$
- VB-M-step: $\theta_0(\theta) \leftarrow \exp \int \log p(Z, x, \theta) Q_0(Z) dZ$

Note: Reduction to EM occurs if $Q_0(\theta) = \delta(\theta - \theta^*)$, in which case $\langle \phi(\theta) \rangle_{\theta(\theta)} = \phi(\theta^*)$.

8.3 Variational Free Energy with Hyperparameters

Remark 8.3 (Free Energy Optimization with Hyper-parameters). The variational free energy can be extended to include hyperparameter optimization:

$$F(\theta_0, \theta_0, \eta) = \int \int Q_0(Z)Q_0(\theta) \log P(X, Z, \theta|D) dZ d\theta \leq P(Z|\eta)$$

The hyper-M step maximizes the current bound with respect to η :

$$\eta) = \arg \max \int \int Q_0(Z)Q_0(\theta) \log P(Z, \theta, \theta|D) dZ d\theta$$

8.4 ARD with Variational Bayesian Methods

Remark 8.4 (Automatic Relevance Determination). ARD is used with variational Bayesian methods for determining latent dimensionality.

Example: For $\gamma_n \sim (\Lambda Z, \psi)$ with prior $M'(0, 1)$ and column-wise prior $\Lambda_{ii} \sim N(0, \alpha_i^{-1})$:

$$F(Q_0, Q_n, \psi, \alpha) = \langle \log P(Y|\Lambda, X) \rangle + \log P(\Lambda|\alpha) + \log P(\Psi) \rangle_{Q_\alpha Q_\nu} + \text{const}$$

Optimization step:

$$\alpha \leftarrow \arg \max \langle \log P(\Lambda|\alpha) \rangle_{Q_\alpha}$$

The hyperparameters α are optimized to cause some divergences as ARD, enabling automatic determination of relevant dimensions.

8.5 Variational Sparse GP Approximations

Definition 8.1 (Sparse Variational GP Methods). Variational sparse GP approximations address the computational cost of Gaussian process inference, which scales as $O(N^3)$ for N data points.

The key idea is to select a smaller set of possible function measurements U at input locations Z such that:

$$P(y|Z, U, x') \approx P(y|x, x', y)$$

GP Notation:

- F : latent GP values for X
- U : latent GP values for Z
- GP prior: $\begin{bmatrix} F \\ U \end{bmatrix} \sim \mathcal{N}\left(0, \begin{bmatrix} K_{xx} & K_{xz} \\ K_{zx} & K_{zz} \end{bmatrix}\right)$

Marginal Likelihood:

$$P(y|X) = \int P(y|F)P(F|U, Z)P(U|Z)dFdU$$

This factorization enables inducing variable methods for efficient inference in GP models.

9 Sigma-Point Filtering and Expectation Propagation

9.1 Sigma-Point Filter

Remark 9.1 (Sigma-Point Filter for Time Series). For the state-space model $P(z_{t+1}|x_{t+1})$ with mean $\hat{Z}_t$ and variance matrix V_t^T :

1. Evaluate the function at multiple sigma points: $f(\hat{Z}_t), f(\hat{Z}_t + \sqrt{V})$
2. For K dimensions, λV satisfies $V_t^T V = \lambda V$
3. Fit a Gaussian to the $2K + 1$ sigma points to estimate:

$$\hat{Z}_t = f(z_{t+1}) + \Sigma$$

4. The fitted Gaussian has parameters:

$$\mathcal{N}\left(\frac{1}{2K+1} \sum f(\delta r), \frac{1}{2K+1} \sum f(\delta r) f(\delta r)^T - \mu \mu^T + \Sigma\right)$$

9.2 Alternative KL Divergence Minimization

Remark 9.2 (The Other KL Direction). In some cases, we minimize a different KL divergence:

$$\arg \min_Q \text{KL}[P(z|x) \| \prod T_z(z|x)^T] = \arg \min - \int p(z|x) \log \prod T_z(z|x) dz$$

This simplifies as:

$$= \arg \min - \sum \int P(z|x) \log \psi_j(z_j|x) dz = \arg \min - \int P(z|x) \log \psi_z(z|x) dz$$

which gives us a cross-entropy form (with $-H(\Sigma) + L(p\pi)$), resulting in:

$$= p(x|z)$$

9.3 Expectation Propagation (EP)

Definition 9.1 (Expectation Propagation Framework). In EP, we assume the posterior factorizes:

$$P(Z|X) = \frac{1}{Z_n} \prod P(Z_i|X_i) \propto \prod_i f_i(z_i)$$

where z_i are not necessarily disjoint. We approximate with sites $\phi(z) \propto \prod_i f_i(z_i)$. EP minimizes:

$$\text{KL}[D\mathcal{P} \| \prod_i] \Rightarrow \min_{\phi} \text{KL} \left[\prod_i f_i(Z_i) \prod_j f_j(Z_j) \middle\| \prod_i f_i^{\text{new}}(Z_i) \prod_j f_j(Z_j) \right]$$

Note: If z_i are truly disjoint, we don't need $\prod f_{jz_i}$. EP reduces to a local KL problem.

9.4 EP Algorithm

Remark 9.3 (Expectation Propagation Algorithm). Given sites $f_1(z_1), \dots, f_W(z_W)$, initialize $\psi_i = \delta_{\text{proj KL}[z_i \| f(z_i)]}$ and $f_j^{\text{new}} = [\text{for } t > 0]$.

Algorithm:

- **Repeat** for $i = 1, \dots, N$ do:
 - **Delete:** $\psi_i(z) = \frac{\phi(z)}{\hat{\ell}_i(z)} = \prod f_j(z)$
 - **Project:** $f_i^{\text{new}}(z_i) = \arg \min_F \text{KL}[f_i(z_i) \psi_i(z_i) \| f_i^{\text{new}}(z_i)]$
 - **Include:** $\psi(z) \leftarrow f_i^{\text{new}}(z_i) \psi_i(z)$
- **Until convergence**

9.5 Message Form in EP

Definition 9.2 (Message Representation in Expectation Propagation). In EP, messages can be expressed as:

$$\psi_i(z_i) := \prod_{j \in \text{trees}} M_{j \rightarrow i}(z_j \wedge z_i)$$

For trees as conditional on z_i , the rest form sub-trees that are disjoint and thus independent, so we compute their product. For loopy graphs, this is only an approximation.

9.6 Moment Matching in Exponential Families

Remark 9.4 (Projection onto Exponential Families). If both $\psi_i(z)$ and $\tilde{\psi}$ are in the same exponential family, with $\phi(x) = \frac{1}{Z(\theta)} e^{T(x)\theta}$:

$$\begin{aligned} \arg \min_{\theta} \text{KL}[\phi(x) \| \tilde{\phi}(x)] &= \arg \min_{\theta} \text{KL} \left[\phi(x) \left\| \frac{1}{Z(\theta)} e^{T(x)\theta} \right\| \right] \\ &= \arg \min_{\theta} - \int p(x) \log \frac{1}{Z(\theta)} e^{T(x)\theta} dx = \arg \min_{\theta} - \int p(x) T(x) \cdot \phi dt + \log Z(\theta) \end{aligned}$$

Taking the derivative with respect to θ :

$$\frac{\partial}{\partial \theta} = - \int dx p(x) T(x) + \frac{1}{Z(\theta)} \frac{\partial}{\partial \theta} \int e^{T(x)\theta} dx = -\langle T(x) \rangle_p + \langle T(x) \rangle_q$$

This is known as **moment matching**: the parameters are chosen so that the marginal expectations of sufficient statistics match.

10 Gaussian Process Approximations and Advanced EP Methods

10.1 Normal Gaussian Process

Definition 10.1 (Gaussian Process with Non-Gaussian Likelihood). Consider the Gaussian process $Y(g)X \sim \mathcal{N}(f(x), \sigma^2 I)$:

$$P(y_1, \dots, y_n, y_i, \dots, y_n) = \mathcal{N}(y_1, \dots, y_n | 0, K) \prod_i \mathcal{N}(y_i | g_i, \sigma_i^2)$$

where the prior is the GP $(f, f_o(\sigma))$ and likelihood is $\prod_i f_i(y_j)$.

If $f(g_j)$ is not Gaussian, approximate by a Gaussian:

$$\tilde{y}(g) \propto f_o(\sigma) \prod \hat{f}_i(y_j) \quad \hat{f} \sim \mathcal{N}(0, Kt \text{ diag}[C[\hat{f}_j^2]])$$

The approximate posterior covariance is:

$$\begin{bmatrix} \mu_{ni} \\ g_j \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} \Sigma_{nn0} & \Sigma_{nri} \\ \Sigma_{in} & K_{ir} \end{bmatrix} \right)$$

Note: $\Sigma_{n,i}$ can be written as $K_{n,i}$ since they are equal. The covariance is $\text{cov}(\mu_{n,j}, y_j) = \text{cov}(g_n + \Sigma_{ni}, y_j) = K_{n,j}$.

Therefore:

$$\psi_{ij}(g_j) = P(g_j | \tilde{\mu}_{ij}) = \mathcal{N}(\tilde{\Sigma}_{n|i} \tilde{\Sigma}_{nn}^{-1} \tilde{\mu}_n, K_i - \Sigma_{i,i}(K_i + \text{diag}(\tilde{\Sigma})_{i,i})^{-1} \Sigma_{i,i})$$

For EP:

$$\arg \min_{\phi} \text{KL}[f_i(y_j) \tilde{\psi}_{ij}(g_j) \| f(g_j) \tilde{\psi}_{ir}(g_j)] \xrightarrow{\text{moment matching}} \hat{f}'_i(g_j) = \frac{\mathcal{N}(\hat{h}(g_j))}{\psi_{ij}(g_j)}$$

with variance:

$$\text{var} = \int f(g_j) \tilde{\psi}_{ij}(g_j) g_j^2 dg - (\bar{\mu}^{\text{new}})^2$$

Prediction:

$$P(y' | x', \mathcal{D}) = \int P(y' | g') \mathcal{N}(g' | K_{y'x}(K_{xx} + \text{diag}(\hat{\sigma}_i^2))^{-1} K_{xw} - K_{xx}(K_{x'x'} + \text{diag}(\hat{\sigma}_i^2))^{-1} K_{xw}) dg'$$

10.2 Exponential Family Factorization

Remark 10.1 (EP Update for Exponential Family Sites). If $f_i(z_i) \propto e^{T(z)\theta_i - \psi(\theta_i)}$ (sites in exponential family), then:

$$q(z) \propto \prod \hat{f}_i^* e^{T(z) \Sigma \theta_i - Z(\psi(\theta_i))}$$

Normalizing, we get:

$$U(z) = e^{T(z) \Sigma \theta_i - \psi(\Sigma \theta_i)}$$

10.3 Unnormalized Log-Sum Approximation

Definition 10.2 (Unnormalized Log-Sum Sites). For approximate sites where $\sum \sum = \sum \theta_i e^{T\Theta\theta}$ with $\theta = \sum \theta_{ij}$ as the natural parameters of $p(z)$:

$$\tilde{\theta}_i = \sum_j \hat{C}_j \quad \text{for } \psi_{ij}(z), \quad \tilde{L}(\theta) = \log \int e^{T\phi\phi} dz \quad (\text{log normalizer})$$

10.4 Unnormalized KL Divergence

Remark 10.2 (Unnormalized KL with Moment Matching). The unnormalized KL divergence is:

$$\text{KL}[\int p||q] = \int p(x) \log \frac{p(x)}{q(x)} dx + \int (q(x) - p(x)) dx$$

This matches the zero moment as well.

10.5 Update Rule for Unnormalized Sites

Definition 10.3 (Computation of Site Updates). For the update:

$$\int \sum \theta e^{T\Theta\theta} \prod \hat{C}_j e^{T(\Theta)\theta_j} = \int f_i(z_i) \prod \hat{C}_j e^{T(\Theta)\theta_j} dz \Rightarrow \hat{C}_j = e^{\hat{B}_i(\theta_n) - E(\theta)}$$

We have:

$$\int f_i(z_i) \prod e^{T(\Theta)\theta_j} dz = e^{\hat{B}_i(\theta_\pi)}$$

where $\hat{B}_i(\theta_\pi)$ is the log normalizer of the fitted exponential family $\hat{f}_i(z_i) \propto f_i(z_i) e^{T(\Theta)\theta_i}$.

10.6 Approximate Log-Likelihood

Remark 10.3 (Log-Likelihood Approximation in EP). The approximate log-likelihood is:

$$\log \int \prod f_i(z_i) \approx \log \int \prod \hat{f}_i(z_i) = \log \left(\prod \hat{C}_j \int e^{T(\Theta)\theta} \right) = E(\theta) + \sum \log \hat{C}_j =: \tilde{L}$$

Since $p(z|x) = \frac{1}{Z} \prod f_i(z_i) = \frac{p(x)}{p(x)} \prod f_i(z_i)$, we have:

$$p(x) = \int \prod f_i(z_i) dz$$

Therefore, $\tilde{L}$ is an approximate log-likelihood.

10.7 Gradient with Respect to Hyperparameters

Definition 10.4 (Hyperparameter Gradient Computation). Let f_i depend on model (hyper) parameters θ' :

$$\nabla_{\theta'} \tilde{L} = \nabla_{\theta'} E(\theta) + \sum \nabla_{\theta'} \log \hat{C}_j \quad \mu = \nabla_{\theta} E(\theta)$$

This expands to:

$$\begin{aligned}
&= \mu \nabla_{\theta'} \theta + \sum \nabla_{\theta'} \log \hat{C}_j^* = \mu \nabla_{\theta'} \theta + \sum (\nabla_{\theta'} \hat{B}_i(\theta_\pi) - \mu \nabla_{\theta'} \theta) \\
&= \mu \nabla_{\theta'} \theta + \sum \left(\mu_i \nabla_{\theta'} \theta_n + \langle \nabla_{\theta'} \log f_i(z_i) \rangle_{\hat{\phi}_i} - \mu \nabla_{\theta'} \theta \right)
\end{aligned}$$

For the $\hat{B}_i$ gradient:

$$\nabla_{\theta'} \hat{B}_i(\theta_{\pi i}) = \nabla_{\theta'} \log \{f_i(z_i) e^{T(\Theta)\theta}\} = \frac{\partial \theta_{\pi i}}{\partial} \hat{B}_i(\theta_{\pi i}) \cdot \nabla_{\theta'} \theta_{\pi i} + e^{\hat{B}_i(\theta_{\pi i})} / \int (\nabla_{\theta'} f_i(z_i)) e^{T(\Theta)\theta} dz$$

This simplifies to:

$$= \mu \cdot \nabla_{\theta'} \theta_{\pi i} + \langle \nabla_{\theta'} \log f_i(z_i) \rangle_{\hat{\phi}_i}$$

where $\langle T(z) \rangle_{\hat{p}_i} = \hat{\mu}_i$ in EP.

10.8 Alpha Divergence

Definition 10.5 (Alpha Divergence and Robust Inference). The alpha divergence is defined as:

$$D_\alpha[P||Q] = \frac{1}{\alpha(1-\alpha)} \int (\alpha P(x) + (1-\alpha)Q(x) - P(x)^\alpha Q(x)^{1-\alpha}) dx$$

Taking limits:

$$\lim_{\alpha \rightarrow 0} D_\alpha(P||Q) = \text{KL}[Q||P] \quad \text{and} \quad \lim_{\alpha \rightarrow 1} D_\alpha(P||Q) = \text{KL}[P||Q]$$

For robust EP with alpha divergence:

$$\text{From} = \arg \max KL[f_i(z_i)^\alpha \hat{f}_i(z_i)^\alpha \tilde{\psi}_{ir}(z_j) || f(z_j) \tilde{\psi}_n(z_j)]$$

Note: For $\alpha < 1$, we have stable results with small updates from $\hat{f}_i(z_i)$.

11 Loopy Inference and Loopy BP Methods

11.1 Dealing with Loops

Remark 11.1 (Damping and Grouping for Loopy Graphs). When the graphical model contains loops, two strategies can improve BP accuracy:

- **Damping:** Update messages with damping factor:

$$M'_{ij}(x_j) = (1 - \alpha)M'_{ij}(x_j) + \alpha \left(\sum f(x_i, x_j) f(x_j) \prod_{k \rightarrow j} M_{k \rightarrow j}(x_j) M_{j \rightarrow i}(x_j) \right)$$

- **Grouping variables into groups:** Aggregate variables into groups to improve accuracy

11.2 Loopy BP as Message-Based Expectation Propagation

Definition 11.1 (Loopy BP as Message-based EP). Loopy BP can be interpreted as message-based expectation propagation where we approximate pairwise factors by products of messages:

$$\psi_{ij}(x_i, x_j) = M_{j \rightarrow i}(x_j) M_{i \rightarrow j}(x_j)$$

The full joint is approximated as:

$$P(x) \approx \frac{1}{Z} \prod_{\text{nodes}} f_i(x_i) \prod_{(x_i, x_j)} \frac{f_{ij}(x_i, x_j)}{1/Z'} = \prod_{\text{nodes}} \left(f_i(x_i) \prod_j \frac{M_{j \rightarrow i}(x_i)}{M_{j \rightarrow i}(x_j)} \right) = \prod_{\text{nodes}} b_i(x_i)$$

EP Interpretation:

- **Deletion:** Remove the pairwise factor:

$$\psi_{ij}(x_i) = f_i(x_i) f_i(x_j) \prod_k M_{k \rightarrow j}(x_j) \prod M_{k \rightarrow j}(x_j) \prod \psi_k(x_j) \prod M_{k \rightarrow j}(x_k)$$

- **Projection:** Find the best message approximation:

$$\psi_{ij}(x_i, x_j) = \int M_{j \rightarrow j}^{\text{new}}, M_{i \rightarrow i}^{\text{new}} = \arg \min \text{KL}[\psi_{ij}(x_i, x_j) \psi_{ij}(x_i, x_j) \| M_{i \rightarrow j}(x_j) M_{j \rightarrow i}(x_j) \hat{c}_{ij}(x_j)]$$

The updated beliefs for variable nodes are:

$$b_i(x_i) \propto M_{j \rightarrow i}^{\text{new}}(x_i) \hat{c}_{ij}(x_j) \propto \sum \left(f_{ij}(x_i, x_j) f_i(x_j) \prod_{k \rightarrow j} M_{k \rightarrow j}(x_j) \right) f(x_j) \prod M_{k \rightarrow j'}(x_i)$$

Thus:

$$M_{ij}^{\text{new}}(x_i) = \sum \left(f_{ij}(x_i, x_j) f_i(x_j) \prod_{k \rightarrow j} M_{k \rightarrow j}(x_j) \right)$$

Important Note: In the tree case, $b_i(x_i) \propto p(x_i)$ is the true marginal. Message-based EP is re-expressed in terms of messages $M_{i \rightarrow j}(x_j)$ instead of sites $f_i(z_i)$, so we don't need to approximate both site and cavity. The cavity is approximated as:

$$\psi_i(z_i) \approx \prod M_{j \rightarrow i}(z_j \wedge z_i) \quad (\text{a product of exponential form})$$

11.3 Loopy BP as Tree-Based Reparameterization

Definition 11.2 (Tree-based Reparameterization of Loopy BP). Loopy BP can also be understood as iteratively reparameterizing the original factorization:

$$P(z) = \frac{1}{Z} \prod_{\text{nodes}} f_i(x_i) \prod_{ij} f_{ij}(x_i, x_j) = P(x_n, y) \prod_{ij} p(x_i | x_j) = \prod_i p(x_i) \prod_{ij} \frac{p(x_i | x_j \epsilon_m(x_j))}{p(x_i) p(x_j)}$$

where we transition from an undirected tree to a DAG representation using parent marginals where $\sum_{x_i} p(x_i, x_j) = p(x_j)$.

Reparameterization Process:

From $P(z) \propto \prod_{ij} f_{ij}(x_i, x_j)$ to $P(x) = \prod_i p(x_i) \prod_{ij} p(x_i | x_j \epsilon_m(x_j))$, we initialize $f_i^0 = f_j^0 = 1$ and iterate over edges:

$$P^n(x_i, x_j) = Z_n^0 f_i^n(x_i) f_{ij}^n(x_i, x_j) f_j^n(x_j) \quad (\text{includes } f_i, f_{ij})$$

Define the updated factors as:

$$f_{ij}^n(x_i, x_j) = \frac{f_{ij}^n(x_i, x_j)}{M_{j \rightarrow i}(x_i)} \quad \text{where we scale down by the message}$$

The propagated messages ensure consistency across the loopy graph structure. For the marginal computations:

$$p(x_i) = \prod_j b_i(x_i), \quad \text{where } b_i(x_i) = f_i(x_i) \prod_j M_{j \rightarrow i}(x_i)$$

and:

$$f_{ij}(x_i, x_j) = \frac{p(x_i, x_j)}{p(x_i)} = (\text{marginalised pairwise})$$

Pseudo-Marginals in Loopy Graphs: While the representation ensures loopy messages remain consistent within the spanning tree, the pseudo-marginals are generally not globally consistent.

For pseudo-marginals to be correct:

$$\sum_{x_j} b(x_i, x_j) = b(x_i), \quad \text{but} \quad \sum_{x_j} \left(b(x_i) \prod_k \frac{b(x_i, x_j)}{b(x_i) b(x_j)} \right) \neq b(x_i)$$

Result: These pseudo-marginals provide valid beliefs for any of the spanning trees in the loopy graph.

Part III: Theoretical Foundations — Convex Optimization and Duality

The approximate inference methods of Part II can be understood through the lens of optimization theory. The Bethe free energy provides a principled objective for loopy belief propagation, while convex duality reveals the fundamental structure underlying all variational inference procedures.

12 Bethe Free Energy and Convex Duality

12.1 Bethe Free Energy on Loopy Consistent Beliefs

Definition 12.1 (Loopy-Consistent Beliefs and Bethe FE). Loopy BP operates on beliefs that satisfy marginal consistency on trees (not globally):

$$\sum_{x_i} b_i(x_i) = 1, \quad \forall i \quad \text{and} \quad \sum_{x_j} b_{ij}(x_i, x_j) = b_i(x_i), \quad \forall i, j \in u(i), x_i$$

The variational free energy is:

$$F(I) = \sum \log P(x_i) - \Sigma + H[\Sigma]$$

The **negative Bethe free energy** is defined as:

$$F_{\text{bethe}}(b) = E_{\text{bethe}}(b) + H_{\text{bethe}}(b)$$

where:

$$P(Y) \propto \prod_i f_i(x_i) \prod_{ij} f_{ij}(x_i, x_j) \propto \prod_i b_i(x_i) \prod_{ij} \frac{b_{ij}(x_i, x_j)}{b_i(x_i)b_j(x_j)}$$

Energy term:

$$E_{\text{bethe}}(b) = \sum_i \sum_{x_i} b_i(x_i) \log f_i(x_i) + \sum_{ij} \sum_{x_i, x_j} b_{ij}(x_i, x_j) \log f_{ij}(x_i, x_j) \quad (\text{using } b_i \text{ as if true marginal})$$

Entropy term:

$$H_{\text{bethe}}(b) = \sum_i H[b_i] - \sum_{ij} \text{KL}[b_{ij} \| b_i b_j] = - \sum_{ij} \sum_{x_i, x_j} b_{ij}(x_i, x_j) \log \frac{b_{ij}(x_i, x_j)}{b_i(x_i)b_j(x_j)}$$

On a tree, $F_{\text{bethe}} = -F$.

12.2 Lagrangian Optimization of Bethe FE

Remark 12.1 (Bethe Free Energy Lagrangian). The Bethe free energy Lagrangian is:

$$\begin{aligned} L = E_{\text{bethe}}(b) + Z_{\text{bethe}}(b) + & \left[\sum a_{ij}(x_i)(-1) + \sum \left(a_{ij}(x_i) \left(\sum_j b_{ij}(x_i, x_j) - b_i(x_i) \right) \right) \right. \\ & \left. + \sum z_{ij}(x_j) \left(\sum_j b_{ij}(x_i, x_j) - b_i(x_j) \right) \right] \end{aligned}$$

Taking derivatives:

$$\frac{\partial L}{\partial b_i(x)} = \log f_i(x_i) - \log b_i(x_i) + \sum \sum \frac{b_{ij}(x_i, x_j)}{b_i(x_i)} + \sum_j - \sum_{j \in \varepsilon(x)} \sum a_{ij}(x) = 0$$

$$\Rightarrow b_i(x_i) \propto f_i(x_i) \prod_{j \in \varepsilon(i)} e^{-z_{ij}(x_i)}$$

For pairwise beliefs:

$$\frac{\partial L}{\partial b_{ij}(x, y)} = \log f_{ij}(x_i, x_j) - \log b_j(x_i, x_j) + \log b_i(y) b_j(y_j) + z_{ij}(x_i) + z_{ij}(y_j) = 0$$

$$\Rightarrow b_{ij}(x_i, x_j) \propto f_{ij}(x_i, x_j) b_i(x_i) b_j(y_j) e^{z_{ij}(x_i)} e^{z_{ij}(x_j)}$$

Comparing to BP: $M_{j \rightarrow i}(x_i) = e^{-z_{ij}(x_i)}$.

Solving for $z_{ij}(x_i)$ by enforcing the consistency constraint $\sum_j b_{ij}(x_i, x_j) = b_i(x_i)$, we have:

$$e^{-z_{ij}(x_i)} \propto \sum_{x_j} f_{ij}(x_i, x_j) b_j(x_j) e^{z_{ij}(x_j)} = \sum_{x_j} f_{ij}(x_i, x_j) f_j(x_j) (\varepsilon(x_i)) e^{-z_{ij}(x_j)}$$

This gives the BP message passing rule.

12.3 Fixed Points of Loopy BP

Definition 12.2 (Bethe FE and Loopy BP Fixed Points). Fixed points of loopy BP are exactly the stationary points of the Bethe tree energy.

12.4 Bethe FE as Second-Order Approximation

Remark 12.2 (Inclusion-Exclusion Approximation). The Bethe free energy is an approximation based on inclusion-exclusion until second order:

$$H_{\text{bethe}}(b) = \sum_i H[b_i] - \sum_{ij} \text{KL}[b_{ij} \| b_i \otimes b_j]$$

The Bethe free energy accounts for interactions between different sites, while variational free energy assumes independence.

12.5 Log Partition Function

Remark 12.3 (Log Partition Function Properties). The log partition function $\Sigma(\theta)$ in exponential family $P(x|\theta) = \exp(\theta^T S(X) - \Sigma(\theta))$ has key properties:

$$\frac{\partial}{\partial \theta} \Sigma(\theta) = \mu, \quad \frac{\partial^2}{\partial \theta^2} \Sigma(\theta) = \text{Var}(S(X))$$

Thus $\Sigma(\theta)$ is convex in θ . The Kullback-Leibler divergence becomes:

$$\frac{\partial \theta}{\partial} = \mathbb{E}_\theta[\theta^T S(X) - \Sigma(\theta)] = \theta^T \mu - \Sigma(\theta)$$

12.6 Negative Entropy and Duality

Definition 12.3 (Negative Entropy as Conjugate). The negative entropy is defined as:

$$\psi(\mu) = \mathbb{E}_\theta[\log P(X|\theta, \psi)] = \mathbb{E}_\theta[e^{\theta^T S(X) - \Sigma(\theta)}] = \theta^T \mu - \Sigma(\theta)$$

Thus:

$$\theta^T \mu = \Sigma(\theta) + \psi(\mu)$$

The negative entropy is dual to the log-partition function.

12.7 Legendre-Fenchel Conjugate

Remark 12.4 (Fenchel Conjugate and Young's Inequality). The Legendre-Fenchel conjugate is defined as:

$$f^*(y) = \sup_{y,x} \langle y, x \rangle - f(x)$$

This is always convex, and Young's inequality states:

$$\langle y, x \rangle \leq f(x) + f^*(y), \quad \forall x, y$$

The Fenchel-Young inequality shows equality holds when:

$$f(x) + f^*(y) = x^T y \iff y \in \partial f(x)$$

Therefore:

$$\mu = \nabla \Sigma(\theta), \quad \theta = \nabla \psi(\mu)$$

13 Convex Duality and Discrete MRF Inference

13.1 KL Divergence and Duality

Definition 13.1 (KL Divergence Bounds and Dual Forms). Consider the KL divergence:

$$\text{KL}[q||p'] = \text{KL}[P(x|\theta_0)P(x|\theta_0)] = \mathbb{E}_\theta[-\log P(x|\theta_0) + \log P(x|\theta)] = -\theta_0^T \mu + \Sigma(\theta_0) + \psi(\mu) \geq 0$$

Therefore:

$$\psi(\mu) = \sup_\theta [\theta^T \mu - \Sigma(\theta)]$$

and continuous functions are reciprocally dual. Thus:

$$\Sigma(\theta) = \sup_\mu [\theta^T \mu' - \psi(\mu')]$$

13.2 Coupled Exponential Family Representations

Remark 13.1 (Factorization in Exponential Family). For $p(x, z) = \exp(\theta^T S(x, z) - \Sigma_{xz}(\theta))$ that is also exponential with changed sufficient statistics, the marginal likelihood is:

$$L(\theta) = p(x|\theta) = \sum_z e^{\theta^T S(x, z) - \Sigma_{xz}(\theta)} = \sum_z e^{\theta^T S_z(z^x) - \Sigma_{xz}(\theta)} = \exp(\Sigma_z(\theta) - \Sigma_{xz}(\theta))$$

The optimization problem becomes:

$$\mu_z^* = \arg \max_\mu [\theta^T \mu_z - \psi(\mu_z)]$$

with the feasibility constraint:

$$\mu = \sum_x \nu(x) S_m(x), \quad \sum_x \nu(x) = 1$$

13.3 Discrete Markov Random Fields

Definition 13.2 (MRF in Exponential Family Form). A discrete MRF can be written as:

$$P(x) = \frac{1}{Z} \prod_i f_i(x_i) \prod_{ij} f_{ij}(x_i, x_j) = \exp \left(\sum_i \sum_k \theta_i(k) S_i(x_i = k) + \sum_{ij} \sum_{k,l} \theta_{ij}(k, l) J(x_i = k)(x_j = l) - \sum_i \sum_k \theta_i(k) S_i(x_i = k) - \sum_{ij} \sum_{k,l} \theta_{ij}(k, l) J(x_i = k)(x_j = l) \right)$$

Natural parameters: $\theta = [\theta_i(k), \theta_{ij}(k, l), \forall i, j, k, l]$

Mean of sufficient statistics: $\mu = [P(x_i = k), P(x_i = k, x_j = l), \forall i, j, k, l]$

For MRF with tree structure:

$$\psi(\mu_T) = \mathbb{E}_\theta \left[\log \prod_i P(x_i) \prod_{ij} \frac{P(x_i, x_j)}{P(x_i)P(x_j)} \right] = - \sum_i H(x_i) + \sum_{ij} I(x_i, x_j)$$

where the first sum is entropy and the second sum is mutual information.

13.4 Non-Convexity of Bethe Entropy

Remark 13.2 (Non-Convexity Issue). The Bethe entropy is not convex:

$$\psi_{\text{bethe}}(\mu_z) = - \sum_i H(x_i) + \sum_{ij} I(x_i, x_j)$$

This presents challenges for optimization.

13.5 Spanning Tree Decomposition

Definition 13.3 (Convex Bound via Spanning Trees). Consider a set of spanning trees T for the MRF, each with its own parameters θ_T, μ_T . By padding entries corresponding to off-tree edges with zero, we assume θ_T has the same dimension as θ .

Suppose we have a distribution ϕ on spanning trees such that $\mathbb{E}_\theta[L_{\sigma T}] = \theta$. Then:

$$\Sigma(\theta) = \tilde{\Sigma}(\mathbb{E}_\theta[L_{\sigma T}]) \leq \mathbb{E}_\theta[\tilde{\Sigma}(\theta_T)]$$

We can obtain a bound to get a good approximation to $\tilde{\Sigma}$:

$$\tilde{\Sigma}(\theta) \leq \mathbb{E}_\theta[\Sigma(\theta_T)] = \Sigma_\theta(\theta)$$

Lagrangian formulation:

$$\mathcal{L} = \mathbb{E}_\theta[\tilde{\Sigma}(\theta_T)] - \lambda^T (\mathbb{E}_\theta[\theta_T] - \theta) = \Sigma_\theta(\theta)$$

Taking derivatives:

$$\frac{\partial \mathcal{L}}{\partial \theta_T} = 0 \Rightarrow \mu_T = \Pi_T(\lambda)$$

where $\Pi_T(\lambda)$ selects multipliers corresponding to elements of θ that are non-zero in tree T .

Dual form:

$$\begin{aligned} \tilde{\Sigma}_\theta(\theta) &= \sup_{\lambda} [\lambda^T \theta + \mathbb{E}_\theta[\Sigma - \psi(\Pi_T(\lambda))]] \\ &= \sup_{\lambda} (\lambda^T \theta + \mathbb{E}_\theta[-\psi(\Pi_T(\lambda))]) \\ &= \sup_{\lambda} \left(\lambda^T \theta + \sum H_\lambda(x_i) - \sum_{ij} I_\lambda(x_i, x_j) \right) \end{aligned}$$

where singleton is always selected in T . This is the **convexified version of Bethe free energy**.

13.6 REINFORCE Trick

Remark 13.3 (Gradient Estimation via REINFORCE). The REINFORCE trick provides a way to estimate gradients:

$$\nabla_\theta F(p, \theta) = \nabla_\theta \int q(z; p) (\log P(z, x | \theta) - \log \mathcal{U}(z; p)) dz$$

$$\begin{aligned}
&= \int \nabla_{\theta} q(z; p) (\log P(z, x | \theta) - \log \mathcal{U}(z; p)) dz + \int q(z; p) \nabla_{\theta} (\log P(z, x | \theta) - \log \mathcal{U}(z; p)) dz \\
&= \int \nabla_{\theta} \log q(z; p) (\log P(x, z | \theta) - \log \mathcal{U}(z; p)) \mathcal{U}(z; p) dz \\
&= \langle \nabla_{\theta} \log \mathcal{U}(z; p) (\log P(x, z | \theta) - \log \mathcal{U}(z; p)) \rangle_{\mathcal{U}(z; p)}
\end{aligned}$$

13.7 Recognition Models and Amortised Inference

Definition 13.4 (Amortised Variational Inference). Instead of learning an optimal variational distribution $\psi(z)$ that depends on x globally, we learn a **recognition model**:

$$\psi(x_i; \theta) \quad \text{where } \theta \text{ is parametrized to depend on } x_i$$

The optimal variational distribution $\psi^*(z)$ becomes:

B recognition model, and this approach is called amortised inference

We learn $f(x_i, \theta)$ instead of ψ .

Part V: Modern Approaches — Neural Networks and Generative Models

Recent advances in machine learning combine neural networks with the approximate inference methods developed throughout this course. The following sections cover parameterized variational inference with deep generative models and contemporary approaches like diffusion models.

14 Deep Learning and Parameterized Variational Inference

14.1 Wake-Sleep Learning Algorithm

Definition 14.1 (Wake-Sleep Learning). The wake-sleep algorithm alternates between two phases for learning both generative and recognition models:

Wake Phase:

- Generative model: $p_\theta(x, z)$ with real data x
- Infer: $z = f(x; \theta)$ with current f
- Update generative parameter: $\theta \leftarrow \theta + \alpha \gamma \nabla_\theta \log f(j, \theta)$

Sleep Phase:

- Recognition model: $q(z|x; f = f(x, \varphi))$, sample $[z, x, z_s]$ from $p(x|z)$
- Update to minimize: $\text{KL}[P(z|z) \| q(z, f(x, \varphi))] \Rightarrow \min -\text{avg}_{z,s} \langle \log f(x, \varphi) \rangle$
- Gradient update: $\Delta \varphi \propto \sum_s (\nabla_s - f(x, \varphi)) \nabla_\varphi f(x, \varphi)$

Alternation:

- **Wake:** Freeze φ , update θ to explain real data
- **Sleep:** Freeze θ , update φ to recover latents on model-generated data

14.2 Variational Autoencoders (VAE)

Definition 14.2 (Variational Autoencoder Architecture). A VAE consists of:

$$P(z) = \mathcal{N}(0, I), \quad P(x|z) = \mathcal{N}(f_{NN}(z; \theta), \sigma^2 I)$$

where $f_{NN}(z; \theta)$ is Gaussian with a neural network for θ .

Posterior approximation (encoder): $q(z|x; f(x, \varphi))$

Variational Objective:

$$\begin{aligned} F(\theta, \varphi) &= \left\langle \sum_z \log P(x|z) + \log P(z) - \log q(z|x; f(x, \varphi)) \right\rangle = \sum_{\text{data}} \log P(x|z_{\text{data}}) - \text{KL}[q(z|x) \| P(z)] \\ &= - \sum_{\text{data}} \left\langle \frac{\|x - g(z)\|^2}{2\sigma^2} \right\rangle_z - \text{KL}[q(z|x) \| P(z)] \end{aligned}$$

where the first term is the reconstruction cost and the second is the regulariser.

14.3 Reparameterization Trick

Remark 14.1 (Gradient Estimation via Reparameterization). If f gives marginal distribution with mean μ and variance Σ_2 for z , we can use:

$$z_i^S = \mu_i + \sigma_i \varepsilon_i^S, \quad \varepsilon_i^S \sim \mathcal{N}(0, 1)$$

The objective becomes:

$$F_{\hat{f}}(\theta, \varphi) = \frac{1}{S} \sum_S \log P(x_i, z_i^S; \theta) - \log q(z_i^S; f(x_i, \varphi))$$

Gradients are computed as:

$$\frac{\partial F_{\hat{f}}}{\partial \theta} = \frac{1}{S} \sum_S \nabla_{\theta} \log P(x_i, z_i^S; \theta)$$

$$\frac{\partial F_{\hat{f}}}{\partial \varphi} = \frac{1}{S} \sum_S \frac{\partial}{\partial z_i} [\log P(x_i, z_i^S; \theta) - \log q(z_i^S; f(x_i, \varphi))] \frac{dz_i^S}{d\varphi} + \frac{\partial}{\partial f(x_i)} \log q(z_i^S; f(x_i, \varphi)) \frac{df(x_i)}{d\varphi}$$

Train θ and φ together, holding z^S fixed.

14.4 Importance-Weighted Variational Bounds

Definition 14.3 (Importance-Weighted Free Energy). The importance-weighted lower bound improves upon the standard ELBO:

$$\gamma(\psi) = \log P(x) - \log \mathbb{E}_{z \sim p_q(x)} [-\log P(x)|z] \geq \log \mathbb{E}_z \left[\frac{P(x, z)}{q(z)} \right] > -F(\psi, \varphi)$$

With K importance samples:

$$\mathcal{L}(k) = \log \mathbb{E}_{z_1, \dots, z_K \sim q} \left[\frac{1}{K} \sum P(x, z_i)/q(z_i) \right] \geq \mathbb{E}_{z_1, \dots, z_K \sim q} \left[\log E/K \sum \frac{P(x, z_i)}{q(z_i)} \right]$$

As $k \rightarrow \infty$, the signal for learning amortised q vanishes as we lose the gradient of z :

$$\nabla_{\varphi} L_k = \mathbb{E}_{z_1, z_K \sim q} \left[\frac{1}{K} \sum (w_k / \sum w_i) \nabla_{\varphi} \log q(z_k) \right], \quad w_k = \frac{P(x, z_k)}{q(z_k)}$$

where $w_k = \frac{P(x, z_k)}{q(z_k)}$ and large $k + 1/K \sum w_i \approx \sum K$ contribute to the gradient.

Note: $\nabla_{\varphi} L_k \propto K \mathbb{E}_{\theta} [\nabla_{\varphi} q(z)] = 0$ as $\mathbb{E}[\nabla_{\varphi} \log q(z)] = 0$.

For small k : The signal dominates at some k , gradient is strong, and $q(z, z)$ learning focuses on high-likelihood region.

14.5 Normalising Flows

Definition 14.4 (Normalising Flows for Flexible Posteriors). Consider a recognition model $q(z)$ implicitly defined by:

$$z_0 \sim p_0(\cdot|x) \leftarrow \text{a fixed, tractable on, e.g. } \mathcal{N}(x, I)$$

Then:

$$z = f_K(f_{K-1}(\dots f_1(z_0))), \quad f_k \text{ is smooth, invertible parametrised by } \varphi$$

The expected log probability transforms as:

$$\mathbb{E}[F(z)]_z = \mathbb{E}[F(f_K(f_{K-1}(\dots f_1(z_0))))]_{z_0}$$

and:

$$\log q(z_0) = \log p_0(f_K^{-1}(\dots f_1(z_0))) - \sum_k \log \left| \frac{\partial f_k}{\partial z_k} \right|$$

Since $z_k = f_k(z_{k-1})$:

$$q(z_{k+1}) = q(f_k^{-1}(z_k)) \left| \frac{\partial z_{k+1}}{\partial z_k} \right|^{-1} = q(f_k^{-1}(z_k)) |\nabla f_k(z_{k-1})|^{-1}$$

Given a sample $z_0^S \sim q_0(\cdot; x)$:

$$F(\varphi, \theta) \propto \frac{1}{2} \sum \log p(x_i, f_K(\dots f_1(z_i^S)); \theta) + H[z_0] + \frac{1}{2} \sum_i \sum_k \log |\nabla f_k(f_{k-1}(\dots f_1(z_i^S)))|$$

15 Diffusion Models and Score-Based Methods

15.1 Diffusion Model Overview

Definition 15.1 (Forward and Reverse Diffusion Processes). A diffusion model combines a forward (noising) process with a learned reverse (denoising) process:

Fixed Forward Diffusion Process: Starting from data x_0 , progressively add Gaussian noise:

$$q(z_t|z_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t}z_{t-1}, \beta_t I)$$

where β_t is a predefined schedule. After T steps:

$$q(z_T) \approx \mathcal{N}(0, I)$$

Markov Chain Structure:

The forward process forms a Markov chain: $z_0 \rightarrow z_1 \rightarrow z_2 \rightarrow \dots \rightarrow z_T$, where each step adds noise. The reverse process learns to denoise: $z_T \rightarrow z_{T-1} \rightarrow \dots \rightarrow z_0$.

15.2 Variational Generative Model (Backward Model)

Definition 15.2 (Reverse Process in Variational Framework). The variational generative model learns to reverse the noising process:

$$p(z_T) = \mathcal{N}(0, I)$$

$$p_\theta(z_{t-1}|z_t) = \mathcal{N}(\mu_\theta(z_t, t), \sigma_t^2 I)$$

$$p_\theta(x|z_0) = \mathcal{N}(\mu_\theta(z_0, 0), \sigma_0^2 I)$$

Evidence Lower Bound (ELBO):

The ELBO decomposes as:

$$\log p(x) \geq \mathbb{E}_{q(z_{1:T}|x)} \left[\log p_\theta(x|z_0) + \sum_{t=1}^T \log \frac{p_\theta(z_{t-1}|z_t)}{q(z_t|z_{t-1})} \right]$$

This is commonly rewritten as a sum of diffusion losses at each timestep t . The forward process posterior at step t is:

$$q(z_t|x) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x, (1 - \bar{\alpha}_t)I)$$

where $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$. The variational model learns to approximate this posterior in reverse.

15.3 Recognition-Parametrized Model (RPM)

Definition 15.3 (Recognition Models as Variational Posterior Approximations). Instead of using a fixed variational distribution, we parameterize the approximate posterior (encoder) as a neural network:

$$q_\phi(z_{t-1}|z_t, x) = \mathcal{N}(\mu_\phi(z_t, t, x), \sigma_\phi^2(z_t, t, x)I)$$

The parameters ϕ of the encoder depend on the observation x and current time step t . This creates an amortized inference mechanism where a single encoder network handles all possible observations.

Joint Training: Both the denoising model p_θ and the recognition model q_ϕ are trained jointly. The recognition model helps guide the reverse process by providing information about the data at each denoising step, improving sample quality and training efficiency compared to unconditional denoising.